

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**
Высшая школа электроники и компьютерных наук
Кафедра системного программирования

**Разработка веб-приложения для электронной приборной панели
автомобиля**

КУРСОВАЯ РАБОТА
по дисциплине «Проектирование и архитектура программных
систем»
ЮУрГУ – 09.03.04.2025.308/1565.КР

Нормоконтролер, Доцент кафедры
СП, к.т.н

_____ М.В. Сухов

“ ” _____ 2025 г.

Научный руководитель:

Доцент кафедры СП, к.т.н

_____ М.В. Сухов

Автор работы:

студент группы КЭ-342

_____ А.Д. Полевик

Работа защищена

с оценкой: _____

“ ” _____ 2025 г.

Челябинск 2025

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное
образовательное учреждение высшего образования
**«Южно-Уральский государственный
университет (национальный
исследовательский университет)»** Высшая
школа электроники и компьютерных наук
Кафедра системного программирования

УТВЕРЖДАЮ
Зав. кафедрой СП

Л.Б. Соколинский

«____».
____.2025

ЗАДАНИЕ

на выполнение курсовой работы
по дисциплине «Программная
инженерия» студенту группы КЭ-342
Полевику Артему Дмитриевичу
(Фамилия Имя Отчество студента)
обучающемуся по направлению
09.03.04 «Программная инженерия»

1. Тема работы

Разработка веб-приложения для электронной приборной панели автомобиля.

2. Срок сдачи студентом законченной работы: 31.05.2025 г.

3. Исходные данные к работе

- 3.1. TypeScript Handbook [Электронный ресурс]. – URL: <https://www.typescriptlang.org/docs/> (дата обращения 18.02.2025)
- 3.2. React documentation [Электронный ресурс]. – URL: <https://react.dev/> (дата обращения 18.02.2025)
- 3.3. Node.js Documentation [Электронный ресурс]. – URL: <https://nodejs.org/en/docs/> (дата обращения 18.02.2025)
- 3.4. Современный учебник JavaScript [Электронный ресурс]. URL: <https://learn.javascript.ru> (Дата обращения 18.02.2025)
- 3.5. Материалы для изучения веб-технологий. [Электронный ресурс] URL: <https://html5book.ru> (Дата обращения 18.02.2025)

4. Перечень подлежащих разработке вопросов

- 4.1. Проанализировать существующие решения и интерфейсы электронных приборных панелей, применяемых в автомобилях;
- 4.2. Выполнить обзор технологий, используемых при создании клиентских веб-интерфейсов и систем передачи данных в режиме реального времени;
- 4.3. Спроектировать архитектуру программной системы, обеспечивающую взаимодействие между эмулятором блока управления и интерфейсной частью;
- 4.4. Реализовать веб-клиент, отображающий параметры автомобиля на основе поступающих телеметрических данных;
- 4.5. Разработать программный эмулятор блока управления, передающий данные с использованием протокола WebSocket;
- 4.6. Провести тестирование системы, включая проверку корректности визуализации параметров и общей стабильности работы.

5. Дата выдачи задания: «___»_____2025 г.

Научный руководитель

М. В. Сухов

Задание принял к исполнению

А. Д. Полевик

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	5
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	8
1.1. Предметная область системы	8
1.2. Анализ аналогичных решений.....	9
1.3. Анализ существующих решений для реализации системы.....	11
2. АНАЛИЗ И ПРОЕКТИРОВАНИЕ СИСТЕМЫ	15
2.1. Определение требований к системе	15
2.2. Диаграмма вариантов использования	16
2.3. Взаимодействие компонентов.....	18
2.4. Проектирование архитектуры	19
2.5. Диаграмма компонентов.....	21
2.6. Диаграмма классов	23
2.7. Диаграмма последовательностей.....	25
2.8. Диаграмма деятельности	26
3. РЕАЛИЗАЦИЯ.....	30
3.1. Программные средства	30
3.2. Реализация пользовательского интерфейса.....	31
3.3. Реализация обработки телеметрических данных.....	33
3.4. Реализация интерфейса блока управления	34
4. ТЕСТИРОВАНИЕ	38
4.1. Функциональное тестирование.....	38
4.2. Юзабилити тестирование	39
ЗАКЛЮЧЕНИЕ	40
ЛИТЕРАТУРА	42

ПРИЛОЖЕНИЯ	44
Приложение А. Спецификация вариантов использования.....	44
Приложение Б. Листинги кода.....	49

ВВЕДЕНИЕ

Актуальность

Автомобильная промышленность стремительно переходит к цифровым решениям, вытесняя аналоговые приборы электронными панелями. Такие интерфейсы позволяют не только более наглядно отображать информацию для водителя, но и адаптировать ее в зависимости от текущих условий. Благодаря этому обеспечивается удобство восприятия, повышение безопасности и улучшение пользовательского опыта.

Разработка электронных приборных панелей требует использования современных подходов к визуализации данных, реализации анимации и организации потокового обмена информацией с минимальной задержкой. Актуальными для этой задачи являются веб-технологии, в частности React [2] и TypeScript [1], а также протоколы реального времени, такие как WebSocket [8].

Создание веб-приложения, имитирующего панель приборов, актуально как с образовательной, так и с прикладной точки зрения. Оно позволяет закрепить навыки разработки интерфейсов, работы с анимацией, потоковыми данными и архитектурой клиент-серверного взаимодействия. Кроме того, такая система может использоваться в демонстрационных, учебных и симуляционных целях.

Постановка задачи

Целью данной курсовой работы является разработка веб-приложения, реализующего цифровую приборную панель автомобиля. Приложение должно отображать ключевые параметры транспортного средства – такие как скорость, обороты двигателя, уровень топлива, температура, напряжение и другие показатели – в реальном времени на основе данных, поступающих от программного блока управления.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Проанализировать существующие решения и интерфейсы

электронных приборных панелей, применяемых в автомобилях;

2. Выполнить обзор технологий, используемых при создании клиентских веб-интерфейсов и систем передачи данных в режиме реального времени;

3. Спроектировать архитектуру программной системы, обеспечивающую взаимодействие между эмулятором блока управления и интерфейсной частью;

4. Реализовать веб-клиент, отображающий параметры автомобиля на основе поступающих телеметрических данных;

5. Разработать программный эмулятор блока управления, передающий данные с использованием протокола WebSocket;

6. Провести тестирование системы, включая проверку корректности визуализации параметров и общей стабильности работы.

Структура и содержание работы

Работа состоит из введения, четырех глав, заключения, списка литературы и приложений. Общий объем составляет 48 страниц. Библиографический список включает 10 наименований.

В первой главе работы выполнен анализ предметной области, посвященной цифровым интерфейсам в автомобилях. Рассматриваются современные тенденции замены аналоговых приборов на электронные панели, а также проведен обзор технологий, применяемых при создании веб-интерфейсов – React, TypeScript, SVG [9], WebSocket. Оцениваются аналогичные решения и обосновывается выбор архитектуры, основанной на программной эмуляции телеметрических данных.

Во второй главе изложено проектирование системы. Сформулированы функциональные и нефункциональные требования, представлена диаграмма вариантов использования, определены основные акторы и сценарии взаимодействия. Разработана архитектура системы, включающая клиентскую часть и блок управления. Приведены диаграммы

компонентов, классов, последовательности и деятельности, отражающие структуру и логику функционирования веб-приложения.

Третья глава посвящена реализации системы. Подробно описаны средства разработки, реализация пользовательского интерфейса приборной панели, обработка входящих телеметрических данных и логика взаимодействия компонентов. Отдельное внимание уделено реализации интерфейса программного блока управления, а также интеграции всех компонентов через WebSocket-соединение.

В четвертой главе проведено тестирование разработанной системы. Проверена корректность отображения параметров, анимации стрелок и работы сигнальных индикаторов. Оценена стабильность соединения и реакция интерфейса на различные сценарии ввода данных. Результаты тестирования сведены в таблицы, подтверждающие работоспособность системы.

В заключении подведены итоги работы. Отмечена успешная реализация интерфейса панели приборов и программного эмулятора блока управления. Также определены направления для дальнейшего развития проекта, включая возможность подключения к реальным источникам данных и расширение функциональности визуализации.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Предметная область системы

Предметной областью проекта является разработка цифровых информационно-управляющих интерфейсов в транспортных средствах, в частности – электронной приборной панели автомобиля. Такие панели предназначены для отображения текущих телеметрических параметров транспортного средства в удобной и наглядной форме для водителя.

В современных автомобилях происходит переход от аналоговых приборов к цифровым решениям, использующим электронные дисплеи и гибкие схемы отображения информации. Это позволяет не только улучшить визуальную читаемость, но и адаптировать интерфейс к условиям движения, отображая дополнительные предупреждения или навигационные данные при необходимости.

Разрабатываемая система представляет собой веб-приложение, визуализирующее параметры автомобиля в режиме реального времени на основе входящих данных от программного блока управления (эмулятора). Блок управления реализован на языке C# с использованием Windows Forms и передает телеметрию через WebSocket-протокол. Клиентская часть, построенная на базе библиотеки React и языка TypeScript, принимает данные, обрабатывает их и отображает в виде графических элементов: стрелочных приборов, цифровых индикаторов и предупреждающих сигналов.

В системе имитируются и отображаются следующие параметры:

- скорость автомобиля;
- обороты двигателя;
- температура охлаждающей жидкости;
- уровень топлива;
- напряжение в бортовой сети;
- текущая передача;

- суммарный пробег.

Проект ориентирован на учебное и демонстрационное применение. Он позволяет моделировать поведение приборной панели, отрабатывать визуализацию и взаимодействие компонентов, осваивать работу с WebSocket и средствами анимации интерфейсов. Архитектура системы спроектирована таким образом, чтобы ее можно было в дальнейшем адаптировать к реальным устройствам через интерфейсы OBD-II или CAN.

Система включает три ключевых компонента:

1. блок управления (ControlBlock) – эмулятор параметров автомобиля;
2. клиентское веб-приложение – визуализирует данные панели;
3. канал передачи данных – реализован через WebSocket-соединение.

Такое решение сочетает в себе простоту, модульность и высокую практическую ценность, делая проект отличной основой для изучения современных технологий визуализации и проектирования интерфейсов в условиях реального времени.

1.2. Анализ аналогичных решений

В современной автомобильной отрасли наблюдается активное внедрение цифровых приборных панелей, способных отображать широкий спектр параметров транспортного средства. Как правило, такие панели являются частью комплексной системы управления автомобилем, интегрированной в его электронную архитектуру. Однако большинство промышленных решений закрыты по своей структуре и не подлежат модификации, что делает их недоступными для обучения, исследований и кастомизации.

Среди открытых и доступных для анализа и экспериментов решений можно выделить три основные категории:

1. Car Dashboard Simulators на базе Arduino и Raspberry Pi – учебные проекты, в которых данные отображаются на LCD-экранах с помощью микроконтроллеров. Отличаются простотой реализации и хорошей наглядностью, но требуют физического оборудования и не масштабируются для более сложных задач [12].

2. Приложения OBD-II (например, Torque, Car Scanner) – мобильные решения, отображающие данные, полученные по Bluetooth от диагностического адаптера. Они используют реальные параметры автомобиля и имеют продуманный пользовательский интерфейс, но требуют наличия автомобиля и оборудования, а также ограничены в возможностях модификации [11].

3. Веб-панели на основе Canvas или SVG – open-source проекты, демонстрирующие цифровую приборную панель прямо в браузере. Обеспечивают хорошую визуализацию, но, как правило, не включают полноценную логику управления, обратную связь или поддержку расширяемой архитектуры.

Сравнительный анализ представленных решений отражен в таблице 1.

Таблица 1 – Сравнение типов решений.

Решение	Преимущества	Недостатки
OBD-II приложения	Реальные данные, готовый интерфейс	Требуют физического автомобиля или адаптера
Canvas-based веб-панели	Работа в браузере, высокая кастомизация	Нет обратной связи, нет логики индикаторов
Эмуляторы приборных панелей на Arduino	Практическая реализация на основе микроконтроллеров	Не масштабируемо, требует физического устройства
Предлагаемое решение	Эмуляция и Web-интерфейс, расширяемость	Не подключается к реальному автомобилю

Разработка в рамках данной курсовой работы сочетает в себе преимущества перечисленных подходов, избегая их недостатков. Система

основана на программной эмуляции блока управления и отображении параметров через веб-интерфейс.

Такой подход обеспечивает:

1. отсутствие необходимости в физическом оборудовании;
2. расширяемую архитектуру с поддержкой новых параметров и индикаторов;
3. кроссплатформенность и независимость от среды исполнения;
4. простоту запуска и использования в учебных целях.

Таким образом, предложенное решение представляет собой оптимальный баланс между функциональностью, доступностью и возможностью дальнейшего развития, что делает его эффективным инструментом как для обучения, так и для построения прототипов в области автомобильных интерфейсов.

1.3. Анализ существующих решений для реализации системы

Разработка программной системы, имитирующей электронную приборную панель автомобиля, требует интеграции нескольких ключевых компонентов, каждый из которых выполняет строго определенную роль в общей архитектуре:

1. эмулятор блока управления – генерирует телеметрические данные;
2. канал обмена данными – обеспечивает передачу параметров в режиме реального времени;
3. клиентская часть (веб-интерфейс) – визуализирует полученные данные;
4. компоненты визуализации и логики – обрабатывают параметры и управляют отображением интерфейса.

Блок управления (ControlBlock)

Источником телеметрических данных в системе выступает десктопное приложение, разработанное на языке C# с использованием

платформы Windows Forms. Программа реализует эмуляцию таких параметров, как скорость, обороты двигателя, температура, напряжение, уровень топлива, текущая передача и пробег.

Особенностью ControlBlock является наличие пользовательского интерфейса, позволяющего вручную моделировать поведение автомобиля: запуск и остановку двигателя, ускорение, торможение, переключение передач и другие действия. Передача данных реализована через WebSocket-соединение, что обеспечивает стабильную двустороннюю связь с веб-клиентом.

Канал передачи данных

В качестве протокола передачи выбран WebSocket, что обусловлено его способностью поддерживать постоянное соединение и передавать данные в режиме реального времени. Это особенно важно при отображении динамических параметров – таких как скорость или обороты двигателя – где задержки и потери данных недопустимы.

В отличие от REST API, WebSocket обеспечивает минимальные накладные расходы, постоянную синхронность и высокую отзывчивость интерфейса.

Клиентская часть (веб-интерфейс)

Клиентская часть разработана с использованием React и TypeScript. Такой стек технологий выбран по следующим причинам:

1. React предоставляет компонентно-ориентированную архитектуру, что упрощает модульность и переиспользуемость кода;
2. TypeScript обеспечивает строгую типизацию, что минимизирует ошибки на этапе компиляции;
3. SVG и HTML позволяют точно и гибко отрисовывать стрелочные приборы и индикаторы;
4. механизм `requestAnimationFrame` [4] реализует плавную анимацию, приближенную к поведению аналоговых приборов.

Отрисовка охватывает: спидометр, тахометр, комбинированный датчик температуры и топлива, индикаторы предупреждений, одометр и индикатор напряжения.

Структура компонентов и логики

Архитектура клиентского приложения условно делится на:

1. компоненты отображения (View): SpeedometerView, TachometerView, DualGaugeView;
2. компоненты логики: useDashboardLogic, WarningIndicators, App;
3. состояния и эффекты: реализуются через хуки useState и useEffect.

Такое разделение приближает проект к архитектуре MVC, где визуализация четко отделена от бизнес-логики. Это способствует улучшению читаемости, сопровождения и расширения проекта.

Особенности интерфейса

Все визуальные элементы реализованы как отдельные компоненты с четко разграниченной ответственностью. Логика получения и обработки параметров полностью инкапсулирована в пользовательском хуке useDashboardLogic. Каждое изменение данных вызывает обновление интерфейса без необходимости перезагрузки страницы.

Развертывание и запуск

Система может быть запущена локально одним действием – через файл START.bat, который одновременно иницирует эмулятор и клиентскую часть. Такой подход делает запуск интуитивным и быстрым, что удобно как при разработке, так и при демонстрации.

Проект не требует серверной инфраструктуры, что упрощает распространение и использование.

Возможности масштабирования

Закладываемая архитектура поддерживает расширение как на уровне клиентской части (новые индикаторы, параметры), так и на уровне источника данных. В будущем программный эмулятор может быть заменен

на физическое устройство, подключенное к CAN-шине или через адаптер OBD-II. Поскольку протокол обмена стандартизирован (WebSocket и JSON), изменения в клиентском интерфейсе при этом минимальны.

Выводы по первой главе

В первой главе был рассмотрен контекст цифровых панелей приборов в автомобилях, включая современные промышленные и учебные решения. Проведен сравнительный анализ мобильных OBD-II-приложений, Arduino-эмуляторов и веб-панелей, выявивший ограничения существующих подходов и подтвердивший целесообразность использования архитектуры с программным эмулятором и веб-интерфейсом.

Такой подход обеспечивает гибкость, удобство тестирования и не требует подключения к реальному оборудованию, что делает его особенно эффективным для учебных задач. Выбор технологий – C# с Windows Forms для блока управления, WebSocket для передачи данных и React с TypeScript для клиентского интерфейса – обоснован с точки зрения надежности, модульности и возможности масштабирования. Проект спроектирован как автономная система с потенциалом дальнейшей интеграции с физическими устройствами.

2. АНАЛИЗ И ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1. Определение требований к системе

По результатам анализа предметной области были сформулированы основные функциональные и нефункциональные требования к проектируемой системе. Разрабатываемое приложение должно эмулировать цифровую приборную панель автомобиля, отображающую параметры транспортного средства на основе данных, поступающих от программного блока управления.

К числу функциональных требований относятся:

1. отображение в режиме реального времени параметров автомобиля: скорости, оборотов двигателя, температуры, уровня топлива, напряжения, пробега и текущей передачи;
2. визуализация параметров с помощью графических компонентов панели приборов: спидометра, тахометра, комбинированного датчика температуры и топлива, одометра и сигнальных индикаторов;
3. отображение предупредительных сигналов при достижении критических значений (перегрев, низкий уровень топлива, низкое напряжение);
4. возможность запуска и остановки двигателя, имитации ускорения и торможения, а также ручного переключения передач со стороны блока управления;
5. плавная анимация стрелок приборов, обеспечивающая реалистичную визуализацию;
6. автоматическое обновление интерфейса без перезагрузки страницы.

К нефункциональным требованиям относятся:

1. передача данных должна осуществляться по протоколу WebSocket для обеспечения низкой задержки и постоянного соединения;

2. клиентская часть должна быть реализована с использованием библиотеки React и языка TypeScript;
3. интерфейс должен быть кросс-браузерным и корректно отображаться при разрешении 1920x1080;
4. запуск системы должен выполняться единым действием через сценарий START.bat, иницирующий и блок управления, и веб-интерфейс;
5. архитектура должна поддерживать расширение функционала и позволять интеграцию с реальными источниками данных (например, через OBD-II или CAN-шину) без значительных изменений клиентской части.

Таким образом, система должна обеспечивать корректную и наглядную визуализацию параметров автомобиля с возможностью имитации поведения в реальном времени, а также обладать гибкой архитектурой для последующего масштабирования.

2.2. Диаграмма вариантов использования

На рисунке 1 представлена диаграмма вариантов использования, отражающая взаимодействие пользователей с системой.

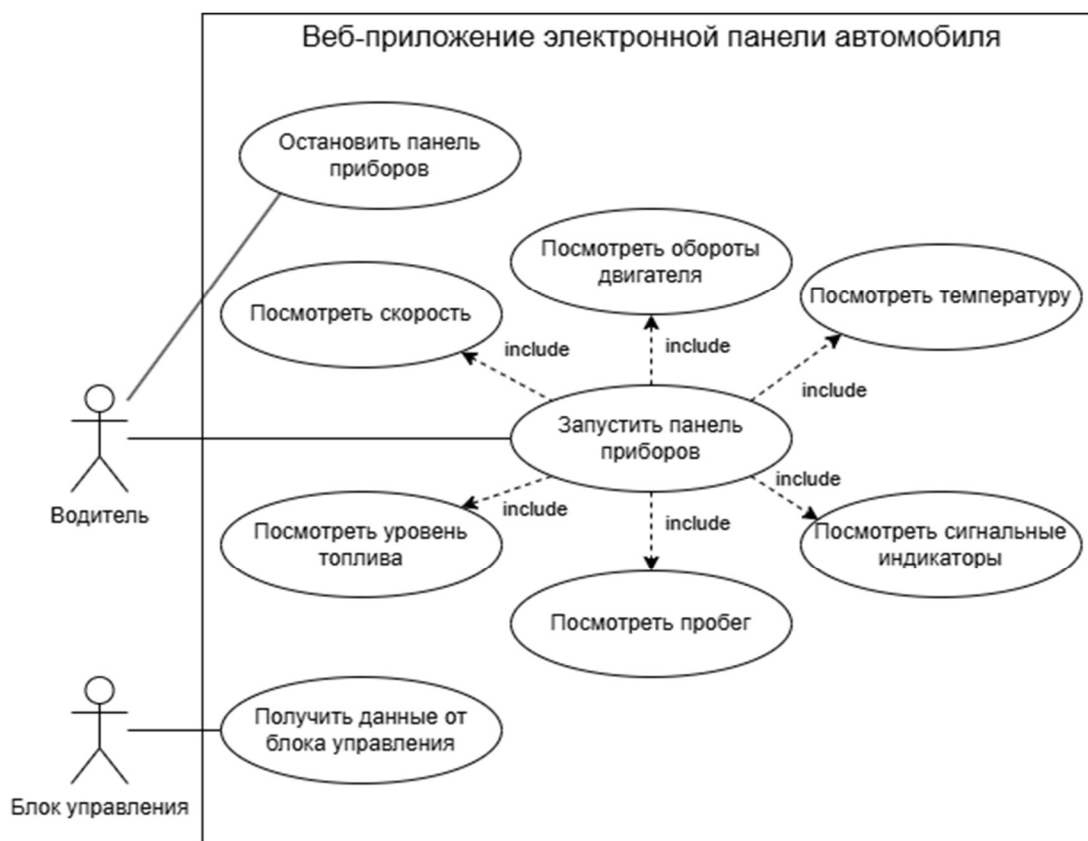


Рисунок 1 – Диаграмма вариантов использования системы

В разработанном приложении выделяются два основных актора: «Водитель» и «Блок управления». Актор «Водитель» представляет конечного пользователя, который инициирует запуск панели приборов, наблюдает за отображаемыми параметрами автомобиля, такими как скорость, обороты двигателя, температура, уровень топлива, напряжение, пробег и сигнальные индикаторы. Также он может завершить работу панели, остановив визуализацию параметров.

Программный актор «Блок управления» (ControlBlock) эмулирует телеметрические параметры автомобиля и автоматически передает их веб-клиенту в виде JSON-сообщений через WebSocket-соединение. Без этого компонента отображение данных на панели становится невозможным. Таким образом, блок управления играет критическую роль в функционировании всей системы, обеспечивая непрерывную подачу данных для визуализации.

Среди основных сценариев взаимодействия можно выделить запуск панели, просмотр параметров движения, отображение предупреждений при достижении критических значений, а также остановку панели приборов. Каждый из этих процессов представлен как отдельный вариант использования, связанный с соответствующим актором. Например, просмотр скорости и оборотов двигателя становится доступен только после активации панели, а отображение тревожных индикаторов связано с текущими значениями параметров, поступающих от блока управления.

Варианты использования объединены логически и отражают основные пользовательские сценарии. Расширенная спецификация каждого прецедента представлена в приложении А.

2.3. Взаимодействие компонентов

Разрабатываемая система состоит из двух основных активных компонентов, взаимодействующих в реальном времени: веб-клиент (интерфейс панели приборов) и блок управления (программный эмулятор параметров автомобиля). Взаимодействие между ними осуществляется по протоколу WebSocket, что позволяет обеспечить передачу телеметрических данных с минимальной задержкой и без необходимости обновления страницы.

Блок управления (ControlBlock) реализован как десктопное приложение на языке C# с использованием Windows Forms. Он играет роль источника данных, формируя и передавая параметры состояния автомобиля, такие как скорость, обороты двигателя, температура, напряжение, уровень топлива, текущая передача и пробег. Эти параметры сериализуются в формате JSON и отправляются клиенту через WebSocket-сервер, встроенный в приложение.

Клиентская часть представляет собой одностраничное веб-приложение, реализованное с использованием библиотеки React и языка

TypeScript. Она устанавливает соединение с WebSocket-сервером и получает телеметрические данные от блока управления. Полученные данные парсятся, проверяются и распределяются по соответствующим компонентам панели. Вся логика обработки событий и обновления состояния сосредоточена в пользовательском хуке `useDashboardLogic`.

Визуальные компоненты панели приборов, такие как `Speedometer`, `Tachometer`, `DualGauge`, `VoltageIndicator` и `WarningIndicators`, получают обновленные значения параметров через `props` и визуализируют их с использованием SVG-графики и механизма `requestAnimationFrame`. Это позволяет добиться плавности анимации и мгновенной реакции интерфейса на изменение параметров.

Система работает в потоковом режиме, не используя постоянное хранилище данных, что соответствует ее симуляционному характеру. Архитектура взаимодействия поддерживает низкую связность компонентов, обеспечивая их независимость и возможность повторного использования или замены без переработки других частей системы.

Таким образом, WebSocket-соединение обеспечивает эффективный канал связи между двумя компонентами, а разделение логики, визуализации и сетевого взаимодействия делает архитектуру гибкой, модульной и расширяемой. Основные фрагменты кода представлены в Приложении Б.

2.4. Проектирование архитектуры

Разрабатываемая система построена по принципу клиент-серверного взаимодействия, где веб-клиент выполняет роль визуализатора, а блок управления — источник телеметрических данных. Архитектура спроектирована с учетом расширяемости, простоты сопровождения и возможности интеграции с реальными устройствами в будущем.

Система состоит из следующих логически независимых компонентов:

1. Клиентское приложение (веб-интерфейс): реализовано с использованием библиотеки React и языка TypeScript. Отвечает за отображение панели приборов, визуализацию параметров в реальном времени и работу с анимацией. Основные визуальные элементы построены на базе SVG-графики, а плавное обновление осуществляется с помощью requestAnimationFrame. Компонент useDashboardLogic инкапсулирует бизнес-логику клиента – прием сообщений, парсинг, распределение данных и обновление состояния через хуки React.

2. Блок управления (ControlBlock): реализован как настольное приложение на C# с использованием Windows Forms. Эмулирует параметры автомобиля (скорость, обороты, температура, топливо и др.) и передает их в веб-интерфейс в формате JSON через WebSocket-соединение. Интерфейс ControlBlock позволяет пользователю вручную изменять параметры, имитируя действия водителя.

3. Канал передачи данных: используется протокол WebSocket, обеспечивающий постоянную двустороннюю передачу данных между ControlBlock и веб-клиентом без необходимости обновления страницы. Это позволяет отображать параметры в реальном времени с минимальными задержками и делает систему отзывчивой.

Обоснование архитектурного подхода:

Компонентный подход, лежащий в основе клиентской части, обеспечивает независимость между визуальными и логическими слоями. Использование React и TypeScript позволяет добиться высокой модульности и типобезопасности. Архитектура ControlBlock реализована автономно и может быть в будущем заменена на реальное устройство без необходимости изменения клиентской логики.

На рисунке 2 представлена диаграмма архитектуры системы, отражающая ключевые компоненты и их взаимодействие.

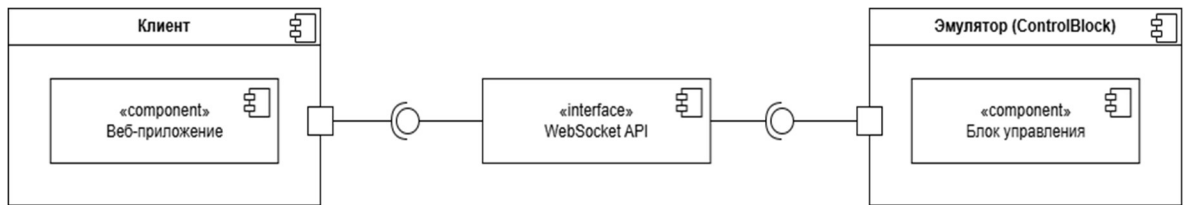


Рисунок 2 – Архитектура программной системы

2.5. Диаграмма компонентов

На рисунке 3 представлена диаграмма компонентов веб-приложения, реализующего электронную приборную панель автомобиля. Диаграмма отражает архитектурное распределение логики между основными модулями системы и их взаимосвязь через интерфейс обмена данными.

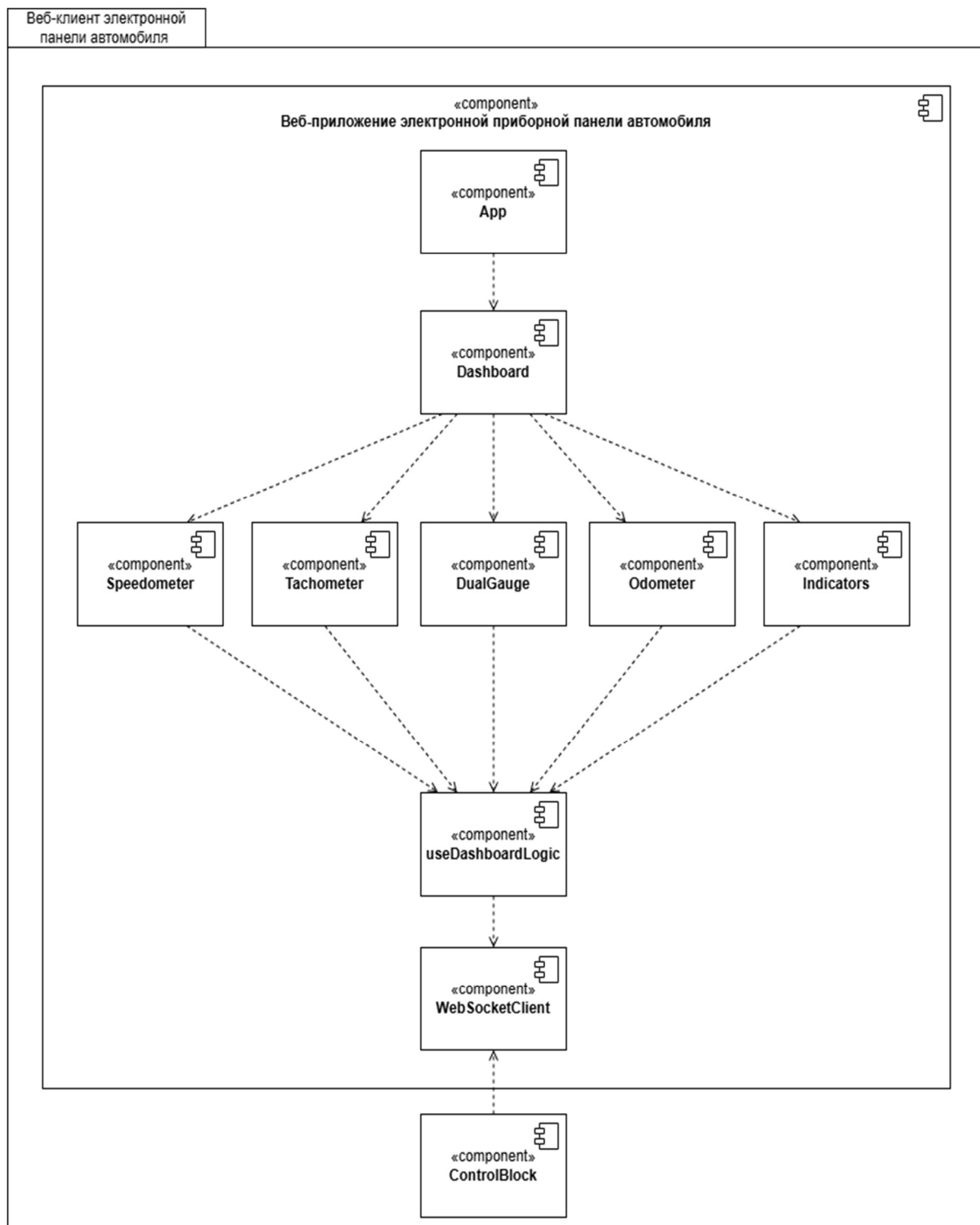


Рисунок 3 – Диаграмма компонентов веб-клиента

Центральным компонентом приложения выступает App, который задает структуру пользовательского интерфейса и инициализирует все вложенные компоненты. Он управляет компонентом Dashboard, где размещены все визуальные элементы приборной панели.

Внутри Dashboard расположены компоненты визуализации, каждый из которых отвечает за отображение конкретного параметра. Компонент Speedometer визуализирует скорость, Tachometer – обороты двигателя, DualGauge – температуру и уровень топлива, Odometer – пробег, а Indicators – сигнальные индикаторы. Все они получают значения через централизованную логику приложения.

Обработка данных реализуется в компоненте useDashboardLogic. Этот модуль устанавливает соединение с WebSocket-клиентом, получает сообщения от блока управления, парсит данные и обновляет состояния, которые передаются в визуальные компоненты через props.

Компонент WebSocketClient устанавливает соединение с внешним источником данных – блоком управления ControlBlock. Последний эмулирует телеметрию автомобиля и передает параметры клиентской части через интерфейс WebSocket API.

Таким образом, архитектура разделена на логические уровни: визуализация, логика и коммуникация с внешними источниками. Компоненты слабо связаны между собой и взаимодействуют через четко определенные интерфейсы, что делает архитектуру гибкой, масштабируемой и удобной для сопровождения.

2.6. Диаграмма классов

Диаграмма классов, представленная на рисунке 4, демонстрирует архитектуру клиентского приложения, реализующего электронную панель приборов. Модель построена на основе принципов объектно-ориентированного проектирования и концепции BCE (Boundary–Control–Entity), которая позволяет разделить структуру приложения на три логических уровня: представление, управление и данные.

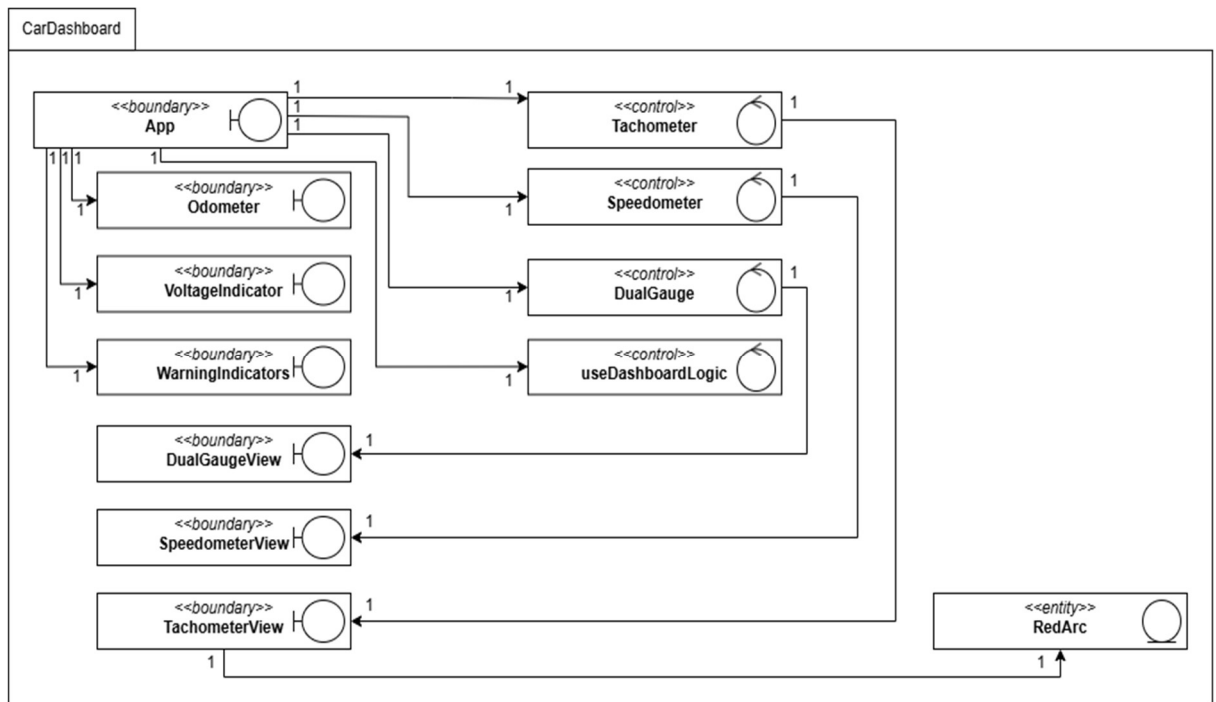


Рисунок 4 – Диаграмма классов

Компоненты Boundary включают визуальные модули, отвечающие за отображение параметров: SpeedometerView, TachometerView, DualGaugeView, Odometer, VoltageIndicator и WarningIndicators. Эти компоненты обрабатывают входные параметры (props) и визуализируют их с использованием SVG-графики.

Control-компоненты (Speedometer, Tachometer, DualGauge) реализуют управляющую логику – сглаживание значений, расчет углов стрелок, передачу значений визуальным модулям. Центральным контроллером выступает пользовательский хук useDashboardLogic, обеспечивающий установку WebSocket-соединения, прием и парсинг телеметрических данных, а также обновление состояний приложения.

В качестве Entity-компонента в системе выделяется RedArc – структура, описывающая геометрию критической зоны тахометра. Этот компонент не взаимодействует напрямую с логикой или представлением, но используется контроллером Tachometer для определения критических значений оборотов.

Все связи между компонентами представлены в виде направленных зависимостей (стрелок), отражающих, какие компоненты используют другие. Диаграмма не включает параметры и методы классов, так как ее цель – визуализация архитектурных связей, а не внутренней реализации. Это особенно уместно для React-приложений, где компоненты реализованы в виде функций и хуков, а не традиционных классов с полями и методами.

2.7. Диаграмма последовательностей

На рисунке 5 представлена диаграмма последовательностей, иллюстрирующая процесс запуска электронной панели и получения первых телеметрических данных от блока управления.

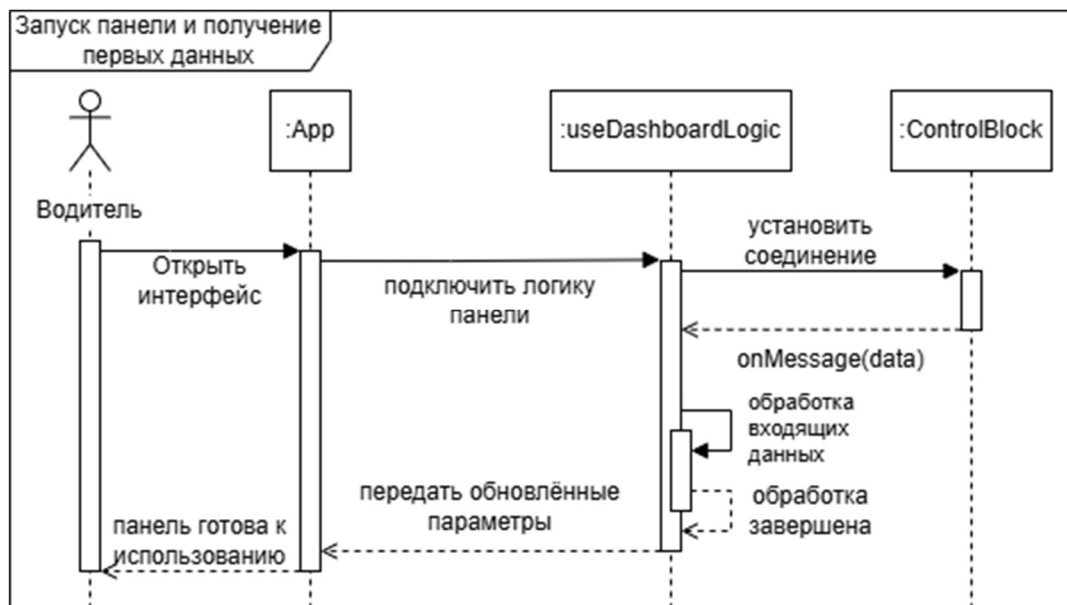


Рисунок 5 – Диаграмма последовательностей запуска панели и получения первых данных

Сценарий начинается с действия актера «Водитель», который инициирует открытие интерфейса, запуская компонент App. Компонент App подключает логику панели через хук useDashboardLogic, в результате чего устанавливается WebSocket-соединение с компонентом ControlBlock, представляющим собой эмулятор блока управления.

После установления соединения ControlBlock начинает передавать телеметрические данные в виде сообщений. Хук useDashboardLogic принимает сообщение через обработчик onMessage(data), затем выполняется разбор и обработка входящих данных. После завершения обработки обновленные параметры передаются обратно в App.

На завершающем этапе интерфейс получает необходимые данные и становится полностью готовым к использованию. Таким образом, взаимодействие между клиентской логикой и источником данных происходит последовательно и синхронно, обеспечивая корректную инициализацию панели.

Диаграмма подчеркивает важные архитектурные особенности: разделение ролей между визуальным компонентом, логическим модулем и эмулятором, а также одностороннюю передачу данных от ControlBlock к клиенту, что соответствует модели передачи телеметрии в реальном времени.

2.8. Диаграмма деятельности

На рисунке 6 представлена диаграмма деятельности, описывающая процесс определения состояния индикаторов панели в зависимости от телеметрических данных, поступающих от блока управления.

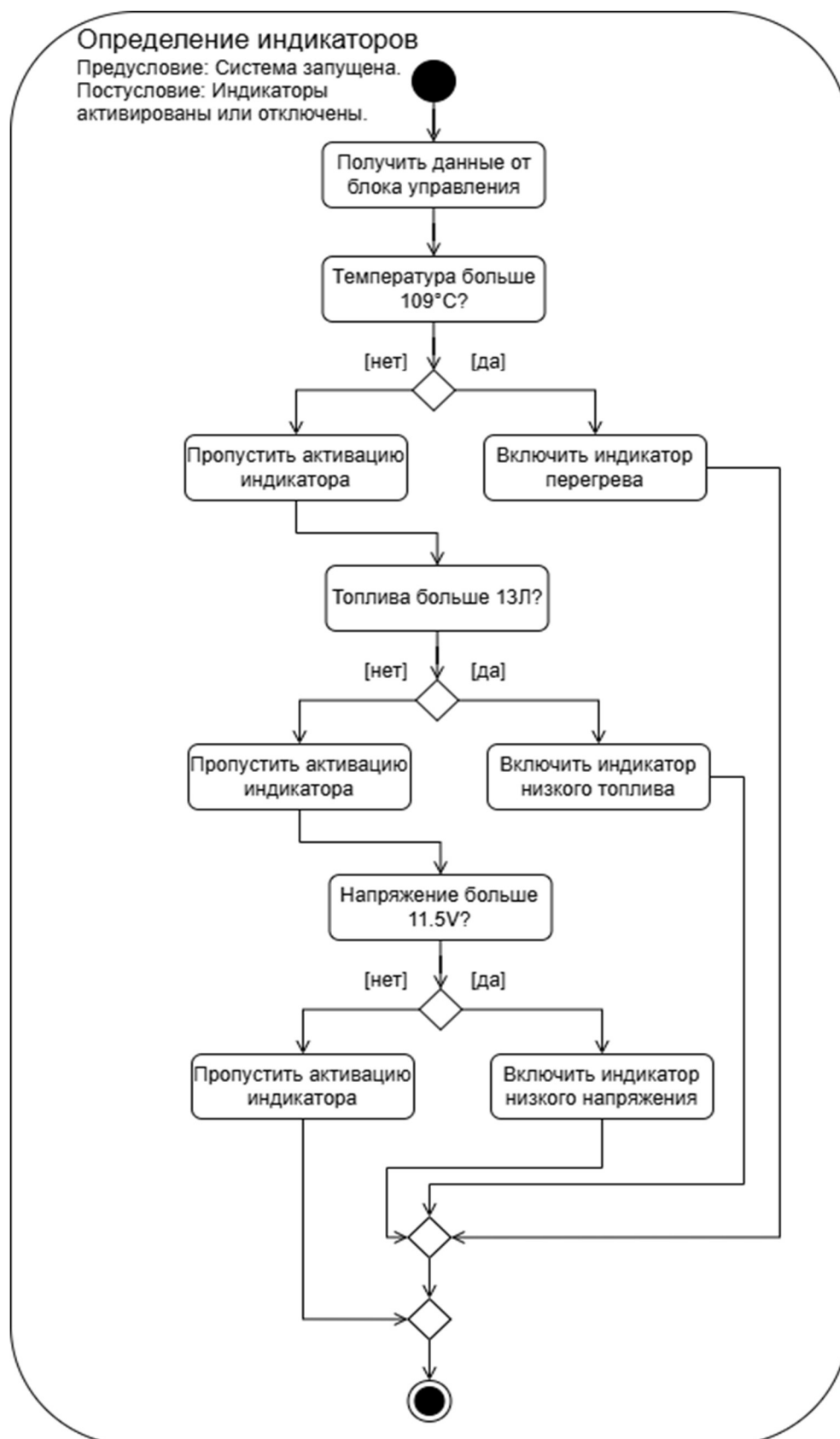


Рисунок 6 – Диаграмма деятельности: определение индикаторов

Процесс начинается с предположения, что система уже запущена. Первым шагом является получение данных от ControlBlock – программного модуля, эмулирующего параметры состояния автомобиля. После этого происходит последовательная проверка трех ключевых показателей:

- температуры двигателя;
- уровня топлива;
- напряжения бортовой сети.

Каждое условие сопровождается ветвлением: если значение параметра превышает заданный порог, активируется соответствующий сигнальный индикатор (перегрев, низкий уровень топлива, низкое напряжение). В противном случае активация индикатора пропускается, и система переходит к следующей проверке.

Логика отображения индикаторов реализуется на клиентской стороне и соответствует принципам предсказуемого и устойчивого поведения интерфейса. Благодаря этой структуре исключается ложное срабатывание индикаторов, а сама диаграмма наглядно демонстрирует порядок принятия решений при визуализации критических параметров.

Завершением процесса является возврат в исходное состояние, при котором система снова готова к приему новых данных и повторной оценке условий отображения.

Вывод по второй главе

Во второй главе был проведен всесторонний анализ и проектирование разрабатываемой системы. На основе анализа предметной области были сформулированы функциональные и нефункциональные требования, определяющие назначение системы и условия ее эксплуатации. Определены основные варианты использования и взаимодействие между ключевыми акторами: водителем и блоком управления.

Построенная диаграмма вариантов использования отразила основные сценарии работы веб-приложения, включая запуск панели, отображение

параметров и получение телеметрических данных. Особое внимание уделено архитектуре приложения – как на уровне клиентской части, так и на уровне эмулятора. Представлена диаграмма компонентов и общая архитектурная схема, демонстрирующие модульность системы и поток данных между ее частями.

Также были спроектированы и проанализированы ключевые диаграммы: классов, последовательностей и деятельности. Они наглядно описывают структуру, поведение и внутренние процессы системы. Такой подход обеспечивает предсказуемость, масштабируемость и надежность архитектурного решения.

В результате можно сделать вывод, что предложенная архитектура позволяет реализовать эффективную и расширяемую симуляционную платформу, пригодную как для учебных задач, так и для будущей интеграции с физическими устройствами.

3. РЕАЛИЗАЦИЯ

3.1. Программные средства

Разработка программной системы выполнена с применением современных кросс-платформенных технологий. Система включает в себя два основных компонента: веб-клиент (реализующий интерфейс электронной панели автомобиля) и блок управления (программный эмулятор телеметрических параметров автомобиля).

Клиентская часть реализована с использованием библиотеки React и языка программирования TypeScript. Такой выбор обеспечивает модульную структуру интерфейса, высокую читаемость и надежность за счет использования статической типизации. Визуальные компоненты панели (спидометр, тахометр, комбинированный датчик температуры и топлива, одометр, сигнальные индикаторы) реализованы с использованием SVG-графики и обновляются с помощью анимационного механизма `requestAnimationFrame`. Для управления состоянием и реакцией на входящие данные используются хуки `useState` и `useEffect`.

Передача данных осуществляется через протокол `WebSocket`, обеспечивающий постоянное двустороннее соединение между клиентом и источником данных. Компонент `useDashboardLogic` отвечает за прием сообщений, парсинг JSON-структуры и распределение параметров по компонентам интерфейса.

Блок управления (`ControlBlock`) реализован как десктопное приложение на языке C# с использованием `Windows Forms`. Его задача – эмулировать значения параметров автомобиля (скорость, обороты двигателя, температура, уровень топлива, напряжение, пробег, передача) и передавать их в веб-клиент. Передача осуществляется в виде сериализованного JSON-объекта через встроенный `WebSocket`-сервер. Управление параметрами осуществляется вручную через элементы интерфейса (кнопки, переключатели), имитирующие действия водителя.

Для разработки компонентов использовались следующие программные средства:

- Visual Studio 2022 – среда разработки блока управления на C#;
- Visual Studio Code – среда разработки клиентской части на TypeScript/React;
- Node.js [3] и npm – управление зависимостями, сборка проекта;
- Windows Terminal/PowerShell – для запуска и отладки;
- браузер Microsoft Edge – тестирование и визуализация интерфейса.

Для упрощения запуска и демонстрации разработан файл START.bat, автоматически запускающий ControlBlock и веб-приложение. Такой подход позволяет инициализировать все компоненты системы единым действием, без необходимости ручного управления процессами.

3.2. Реализация пользовательского интерфейса

Пользовательский интерфейс панели приборов разработан как одностраничное веб-приложение (SPA) с использованием библиотеки React и языка программирования TypeScript. Такой подход позволил реализовать модульную архитектуру, гибкую структуру компонентов и высокую отзывчивость интерфейса в реальном времени.

Визуальная часть представляет собой совокупность компонентов, отображающих параметры автомобиля. Отрисовка всех графических элементов (стрелок, шкал, цифровых индикаторов) осуществляется с использованием технологии SVG. Для анимации применен встроенный браузерный механизм requestAnimationFrame, обеспечивающий плавность перемещения стрелок и своевременное обновление значений.

Интерфейс содержит следующие визуальные элементы:

- спидометр – отображает текущую скорость;
- тахометр – показывает обороты двигателя;

- комбинированный датчик – объединяет отображение температуры и уровня топлива;
- одометр – отображает пробег;
- вольтметр – отображает напряжение бортовой сети;
- индикаторы предупреждений – сигнализируют о перегреве, низком топливе, снижении напряжения.

Все эти элементы размещаются на фоне, имитирующем приборную панель, и масштабируются для корректного отображения при разрешении 1920x1080. Основная структура интерфейса задана в компоненте App, который формирует каркас панели и координирует работу остальных компонентов.

Каждый визуальный компонент разделен на две части: логическую и отображающую. Логическая часть отвечает за получение, интерполяцию и сглаживание параметров, визуальная – за отрисовку. Например, компонент Speedometer обрабатывает данные скорости, а SpeedometerView строит графику по переданным значениям. Такой подход соответствует принципу разделения ответственности и повышает читаемость и масштабируемость проекта.

Для простых элементов, таких как VoltageIndicator, Odometer и WarningIndicators, логика и визуализация объединены в одном компоненте, что оправдано их меньшей сложностью.

Данные в интерфейс поступают из пользовательского хука useDashboardLogic, который устанавливает WebSocket-соединение с эмулятором блока управления. Хук получает сообщения в формате JSON, извлекает параметры и обновляет состояния, передаваемые в соответствующие визуальные компоненты.

На рисунке 7 представлен внешний вид реализованного интерфейса во время работы системы.

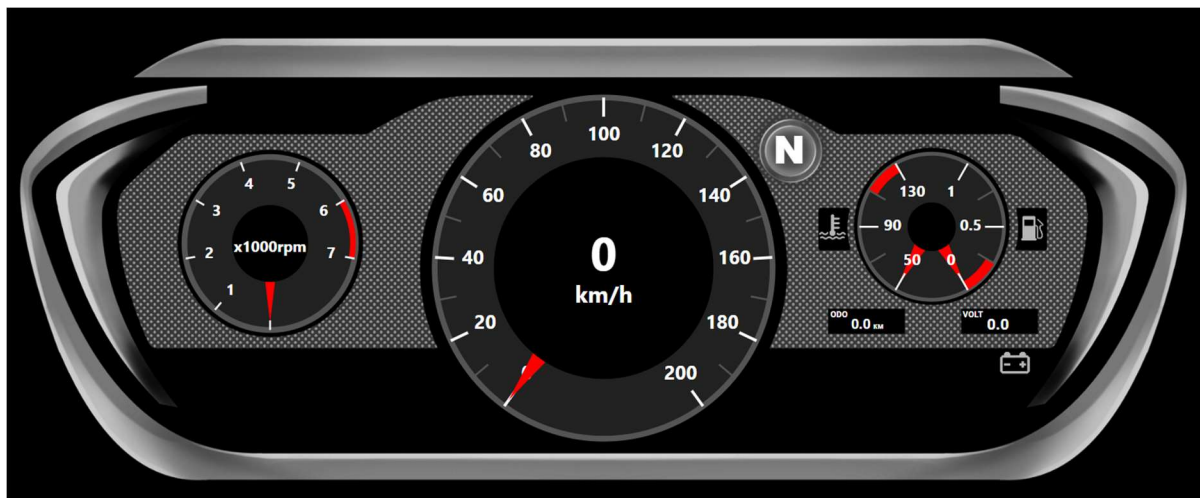


Рисунок 7 – Интерфейс панели приборов

Такое компонентное построение интерфейса с применением SVG-графики, анимации и современных веб-технологий обеспечивает высокую реалистичность отображения, легкость в расширении и стабильную работу системы в реальном времени при разрешении 1920x1080.

3.3. Реализация обработки телеметрических данных

Обработка телеметрических параметров автомобиля реализована с использованием пользовательского хука `useDashboardLogic`. Он выполняет центральную роль в управлении состоянием интерфейса и получении данных от блока управления.

После запуска приложения устанавливается соединение с WebSocket-сервером, встроенным в `ControlBlock`. При получении JSON-сообщений, содержащих значения параметров автомобиля (скорость, обороты, температура, напряжение, уровень топлива, пробег и передача), хук производит их разбор и сохраняет с помощью React-хуков `useState`. Эти

состояния затем передаются визуальным компонентам, отвечающим за отображение значений на панели.

Одним из ключевых элементов реализации является сглаживание параметров – особенно тех, которые визуализируются с помощью стрелок. Для этого применяется механизм `requestAnimationFrame`, обеспечивающий плавную интерполяцию между предыдущим и текущим значением параметра. Такой подход повышает реалистичность отображения и снижает резкие скачки интерфейса при изменении данных.

Дополнительная логика касается активации сигнальных индикаторов. На основании текущих значений температуры, напряжения и уровня топлива компонент `WarningIndicators` определяет необходимость отображения предупреждающих значков (например, перегрев при температуре выше 109 °C или низкий уровень топлива ниже 13 %).

Таким образом, обработка телеметрических данных реализована по модульному принципу. Центральный компонент отвечает за прием и интерпретацию данных, в то время как визуальные элементы остаются изолированными и получают только готовые к отображению значения. Это обеспечивает надежность, масштабируемость и легкость сопровождения всей системы.

3.4. Реализация интерфейса блока управления

Блок управления (`ControlBlock`) реализован как Windows Forms-приложение на языке C#. Его задача – эмулировать телеметрию автомобиля и отправлять параметры клиентскому приложению по WebSocket-соединению.

На рисунке 8 показан внешний вид интерфейса блока управления.

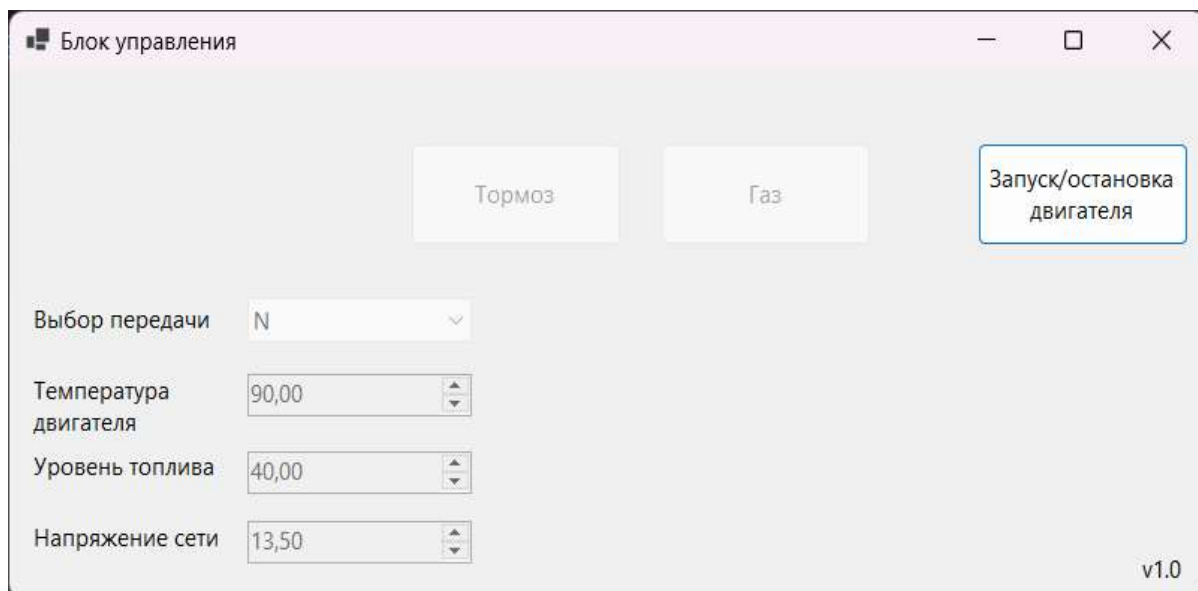


Рисунок 8 – Интерфейс блока управления

Элементы интерфейса включают:

- Кнопки (Button): «Газ», «Тормоз», «Запуск/остановка двигателя» – имитируют действия водителя.
- ComboBox: используется для выбора передачи. Доступны только фиксированные значения: N, 1, 2, 3, 4, 5.
- NumericUpDown: применяются для задания температуры двигателя, уровня топлива и напряжения бортовой сети.
- Label: отображают подписи к полям и версию приложения.

Все элементы распределены по группам, обеспечивая интуитивную навигацию. Интерфейс не перегружен визуально, благодаря чему остается простым и понятным.

Функциональные особенности и валидация:

- Перед запуском двигателя остальные элементы управления блокируются, предотвращая случайные действия.
- Температура двигателя ограничена сверху (130 °C);
- Пробег может только расти – значение нельзя уменьшить;

- Напряжение сети и уровень топлива – могут быть заданы в пределах безопасных значений (валидация встроена через свойства NumericUpDown);
- ComboBox разрешает только выбор из фиксированного списка передач;
- Кнопка запуска двигателя активирует передачу данных, в противном случае параметры не отправляются.

Передача параметров осуществляется автоматически с помощью встроенного таймера. Значения сериализуются в формате JSON и передаются через WebSocket-сервер, встроенный в ControlBlock.

Такой интерфейс позволяет точно и гибко управлять состоянием симулируемого автомобиля, обеспечивая реалистичное поведение для целей тестирования и демонстрации панели приборов.

Вывод по третьей главе

В третьей главе рассмотрена реализация всех компонентов программной системы, включая веб-клиент, блок управления и механизм обмена телеметрическими данными. Клиентская часть реализована с использованием библиотеки React и языка TypeScript, визуализация выполнена на основе SVG-графики с применением компонентной архитектуры. Это обеспечило модульность, гибкость и точность отображения.

Подробно описан процесс реализации пользовательского интерфейса, в том числе взаимодействие между визуальными и логическими компонентами, передача данных через WebSocket и применение requestAnimationFrame для сглаживания анимации.

Рассмотрена логика обработки телеметрических параметров, включая подключение к WebSocket-серверу, прием и интерпретацию данных, а также обновление состояния интерфейса в реальном времени.

Интерфейс программного блока управления, разработанный с использованием Windows Forms, включает элементы ручного управления

параметрами, а также реализует валидацию вводимых значений и синхронизацию с клиентом.

Таким образом, глава демонстрирует полноту реализации системы и ее техническую готовность к дальнейшему расширению – в том числе возможной интеграции с реальными источниками данных без необходимости переработки архитектуры клиентской части.

4. ТЕСТИРОВАНИЕ

4.1. Функциональное тестирование

Функциональное тестирование [6] – это процесс проверки программного продукта на соответствие его функциональным требованиям. Оно включает в себя тестирование функций, взаимодействия с пользователем, обработки данных и других аспектов, чтобы убедиться, что программа работает правильно и выполняет запланированные задачи.

Результаты проведения функционального тестирования представлены в таблице 2.

Таблица 2 – Результаты функционального тестирования.

№	Название теста	Шаги выполнения	Ожидаемый результат	Тест пройден
1	Получение параметров от ControlBlock	1. Запустить START.bat 2. Убедиться в появлении интерфейса панели и блоке управления	Панель приборов активируется и ожидает запуска с блока управления.	Да
2	Запуск двигателя с нулевой скоростью	1. Нажать «Старт двигателя» 2. Убедиться, что стрелки показывают холостой режим	Скорость = 0, обороты = холостой ход (~800 об/мин)	Да
3	Одновременное изменение нескольких параметров	1. В ControlBlock одновременно изменить скорость, обороты и температуру	Все значения корректно обновляются в интерфейсе, анимации работают плавно	Да
4	Обновление интерфейса без перезагрузки	1. Изменить любые параметры в ControlBlock	Интерфейс панели обновляется автоматически, без необходимости перезагрузки страницы	Да
5	Отображение скорости	1. В блоке управления нажать и удерживать кнопку «Газ» 2. Проверить положение стрелки и цифровое значение	Спидометр отображает изменения скорости; стрелка анимировано сместится.	Да

6	Отображение предупреждений (низкий уровень топлива)	1. Установить значение топлива ниже 13% 2. Проверить отображение индикатора топлива	Активируется соответствующий индикатор на панели.	Да
7	Остановка двигателя	1. Нажать "Заглушить двигатель" в блоке управления 2. Проверить, что стрелки обнулились и индикаторы отключены	Все параметры сбрасываются, отображение становится неактивным	Да
8	Реакция на аномальные значения	1. Установить температуру выше 130°C	Данные игнорируются или отображается безопасное значение, ошибки не возникает	Да
9	Реакция на низкое напряжение	1. Установить напряжение ниже 11.5 В 2. Проверить активность индикатора	Активируется индикатор низкого напряжения	Да

4.2. Юзабилити тестирование

Проводилось внутреннее тестирование интерфейса на предмет понятности, доступности и логики размещения элементов.

Критерии оценки:

1. интуитивность отображения приборов и индикаторов;
2. логичность визуального поведения стрелок;
3. мгновенная реакция на изменение параметров;
4. удобство взаимодействия с ControlBlock.

Вывод: Все элементы интерфейса панели визуально различимы, реагируют предсказуемо и в реальном времени. Отображение параметров происходит без задержек. Пользователь не требует дополнительного обучения для взаимодействия с системой.

Вывод по четвертой главе

Все тесты завершились успешно, что говорит о высокой стабильности и готовности системы к дальнейшему применению.

ЗАКЛЮЧЕНИЕ

В рамках данной курсовой работы было разработано веб-приложение, имитирующее электронную панель приборов автомобиля. Целью проекта являлось создание визуального интерфейса, отображающего телеметрические параметры в реальном времени на основе данных, поступающих от программного блока управления.

В ходе работы были решены следующие задачи:

1. Проведен анализ предметной области, включая обзор современных решений в сфере цифровых приборных панелей, применяемых как в серийных автомобилях, так и в учебных проектах.
2. Определены функциональные и нефункциональные требования к разрабатываемой системе.
3. Спроектирована архитектура системы, включающая блок управления (ControlBlock), клиентское веб-приложение и механизм передачи данных на основе WebSocket.
4. Реализованы основные компоненты клиентской части с использованием библиотеки React и языка TypeScript. Визуализация выполнена с применением SVG-графики и анимации стрелочных элементов.
5. Создан и интегрирован десктопный эмулятор параметров автомобиля на C# (Windows Forms), обеспечивающий генерацию данных и передачу в реальном времени.
6. Проведено функциональное тестирование всех ключевых сценариев, включая отображение скорости, оборотов, предупреждающих индикаторов и работу при пограничных значениях.

Разработанная система демонстрирует модульную архитектуру и готова к масштабированию. Благодаря использованию современных веб-технологий и компонентного подхода интерфейс легко адаптируется и

расширяется. Передача данных по WebSocket обеспечивает реалистичную имитацию взаимодействия с реальным транспортным средством.

Перспективы развития проекта

В дальнейшем возможно расширение функциональности проекта, включая:

1. Подключение к реальному автомобилю через адаптер OBD-II или CAN-шину;
2. Расширение набора отображаемых параметров (например, расход топлива, температура масла);
3. Добавление звуковых и световых индикаторов;
4. Визуальное оформление панели под разные модели автомобилей;
5. Адаптация интерфейса под сенсорные экраны и мобильные устройства.

Проект может быть полезен для учебных целей, демонстрации архитектурных подходов в frontend-разработке и как основа для дальнейших инженерных экспериментов.

Исходный код веб-приложения доступен по адресу:
<https://github.com/Berkyla/car-dashboard>

ЛИТЕРАТУРА

1. TypeScript Handbook. [Электронный ресурс] URL: <https://www.typescriptlang.org/docs/> (дата обращения 18.02.2025 г.).
2. React documentation. [Электронный ресурс] URL: <https://react.dev/> (дата обращения 18.02.2025 г.).
3. Node.js Documentation. [Электронный ресурс] URL: <https://nodejs.org/en/docs/> (дата обращения 18.02.2025 г.).
4. Современный учебник JavaScript. [Электронный ресурс] URL: <https://learn.javascript.ru> (дата обращения 18.02.2025 г.).
5. Материалы для изучения веб-технологий. [Электронный ресурс] URL: <https://html5book.ru> (дата обращения 18.02.2025 г.).
6. Юрченко А.Н. «Функциональное тестирование как одна из методологий тестирования программного обеспечения». Вестник современных исследований. [Электронный ресурс] URL: <https://elibrary.ru/item.asp?id=32535053> (дата обращения: 16.04.2025 г.).
7. Лисовенко А.Ю. «Особенности разработки интерактивных пользовательских интерфейсов с использованием React JS». Информационные технологии, системный анализ и управление (ИТСАУ-2021): сб. тр. XIX Всерос. науч. конф. молодых ученых, аспирантов и студентов, Ростов-на-Дону – Таганрог, 2021. [Электронный ресурс] URL: <https://elibrary.ru/item.asp?id=48597875> (дата обращения: 18.02.2025 г.).
8. WebSocket Handbook. [Электронный ресурс] URL: <https://websocket.org/> (дата обращения: 26.02.2025 г.).
9. W3C «Scalable Vector Graphics (SVG) 2». [Электронный ресурс] URL: <https://www.w3.org/TR/SVG2/> (дата обращения: 20.02.2025 г.).
10. Брусникин Г. Н., Соколова Н. Ю. «Разработка UML-моделей при проектировании информационных систем: учебное пособие». [Электронный ресурс] URL: <https://e.lanbook.com/book/461570> (дата обращения: 27.03.2025 г.)

11. CAR SCANNER ELM OBD2. [Электронный ресурс] URL: <https://www.carscanner.info/ru/?ysclid=mboo0s9r93160719159> (дата обращения: 07.05.2025 г.)

12. SIM Dashboard. [Электронный ресурс] URL: https://www.stryder-it.de/simdashboard/ets2_ats?show_desktop_mode=false (дата обращения: 07.05.2025)

ПРИЛОЖЕНИЯ

Приложение А. Спецификация вариантов использования

Таблица 1 – Реализация прецедента «Запустить панель приборов»

Прецедент: Запустить панель приборов
ID: 1
Краткое описание: Водитель запускает панель приборов, в результате чего начинается отображение параметров автомобиля
Главные актеры: Водитель
Второстепенные актеры: Нет
Предусловия: Панель приборов не активна. Блок управления запущен.
Основной поток: 1. Водитель открывает веб-интерфейс. 2. Система инициализирует компонент App. 3. Компонент useDashboardLogic устанавливает WebSocket-соединение с блоком управления. 4. Блок управления передает поток данных (скорость, температура, топливо и др.). 5. Система начинает отображение параметров в виде анимированных стрелок и значений. 6. Компоненты визуализации (Speedometer, Tachometer и др.) обновляют свое состояние.
Постусловие: Панель активирована. Показатели параметров визуализируются на экране.
Альтернативные потоки: Нет.

Таблица 2 – Спецификация прецедента «Посмотреть скорость»

Прецедент: Посмотреть скорость
ID: 2
Краткое описание: Позволить пользователю отслеживать актуальное значение скорости автомобиля через интерфейс панели приборов
Главные актеры: Водитель
Второстепенные актеры: Нет
Предусловия: Панель приборов запущена
Основной поток: 1. Система получает значение скорости от блока управления. 2. Компонент спидометра отображает значение с помощью стрелки и цифрового индикатора. 3. Водитель видит текущую скорость на панели.
Постусловия: Пользователь видит актуальное значение скорости в цифровом виде и на шкале со стрелкой.
Альтернативные потоки: Нет

Таблица 3 – Реализация прецедента «Просмотр оборотов двигателя»

Прецедент: Просмотр оборотов двигателя
ID: 3
Краткое описание: Отобразить текущие обороты двигателя на тахометре.
Главные актеры: Водитель
Второстепенные актеры: Нет
Предусловия: Панель приборов запущена
Основной поток: 1. Водитель наблюдает значение тахометра. 2. Система визуализирует текущие обороты двигателя на компоненте тахометра.
Постусловие: Обороты отображаются на шкале тахометра и, при необходимости, в цифровом виде.
Альтернативные потоки: Нет

Таблица 4 – Реализация прецедента «Просмотр температуры двигателя»

Прецедент: Просмотр температуры двигателя
ID: 4
Краткое описание: Предоставить водителю информацию о текущей температуре двигателя
Главные актеры: Водитель
Второстепенные актеры: Нет
Предусловия: Панель приборов запущена
Основной поток: 1. Водитель обращает внимание на комбинированный датчик. 2. Система отображает текущую температуру двигателя на графическом индикаторе.
Постусловие: Температура двигателя отображается на комбинированном датчике температуры и уровня топлива.
Альтернативные потоки: Нет

Таблица 5 – Реализация прецедента «Просмотр уровня топлива»

Прецедент: Просмотр уровня топлива
ID: 5
Краткое описание: Предоставить водителю информацию о текущем уровне топлива в баке
Главные актеры: Водитель
Второстепенные актеры: Нет
Предусловия: Панель приборов запущена

Окончание таблицы 5 приложения А

Основной поток:
1. Водитель наблюдает за шкалой топлива на приборной панели.
2. Система отображает текущую температуру двигателя на графическом индикаторе.
Постусловие: Уровень топлива визуализируется на комбинированном индикаторе
Альтернативные потоки: Нет

Таблица 6 – Реализация прецедента «Просмотр одометра»

Прецедент: Просмотр одометра
ID: 6
Краткое описание: Показать водителю текущее значение пробега автомобиля
Главные актеры: Водитель
Второстепенные актеры: Нет
Предусловия: Панель приборов запущена
Основной поток:
1. Водитель обращает внимание на цифровой дисплей одометра.
2. Система отображает актуальное значение пробега в километрах.
Постусловие: Значение пробега отображается на одометре
Альтернативные потоки: Нет

Таблица 7 – Реализация прецедента «Просмотр индикаторов предупреждения»

Прецедент: Просмотр индикаторов предупреждения
ID: 7
Краткое описание: Водитель наблюдает за индикаторами тревоги (низкий уровень топлива, перегрев двигателя, низкое напряжение), которые автоматически активируются при достижении критических значений параметров.
Главные актеры: Водитель
Второстепенные актеры: Нет
Предусловия: Панель приборов запущена

Основной поток:
1. Система получает данные от блока управления.
2. Происходит анализ параметров: температуры, топлива, напряжения.
3. Если один из параметров выходит за допустимые границы:
3.1. При температуре $> 109^{\circ}\text{C}$ – активируется индикатор перегрева.
3.2. При уровне топлива $< 13\%$ – активируется индикатор низкого топлива.
3.3. При напряжении $< 11\text{ В}$ – активируется индикатор низкого напряжения.
4. На интерфейсе панели соответствующий индикатор отображается водителю.
5. После стабилизации параметров индикатор автоматически исчезает.
Постусловия: Индикаторы отображаются в соответствии с состоянием параметров.
Альтернативные потоки: Нет

Таблица 8 – Реализация прецедента «Остановить панель приборов»

Прецедент: Остановить панель приборов
ID: 8
Краткое описание: Пользователь завершает сеанс работы панели приборов, что приводит к остановке визуализации параметров и отключению соединения с блоком управления
Главные актеры: Водитель
Второстепенные актеры: Нет
Предусловия: Приборная панель запущена
Основной поток:
1. Водитель нажимает кнопку «Заглушить двигатель» в интерфейсе блока управления.
2. Блок управления отправляет сигнал завершения работы.
3. Панель прекращает прием и отображение параметров.
4. Все стрелки обнуляются, индикаторы гаснут.
5. Интерфейс фиксирует остановленное состояние.
Постусловия: Приборная панель деактивируется, визуальные параметры сбрасываются
Альтернативные потоки: Нет

Таблица 9 – Реализация прецедента «Получить данные от блока управления»

Прецедент: Получить данные от блока управления
ID: 9
Краткое описание: Система принимает телеметрические параметры автомобиля от блока управления через WebSocket-соединение для дальнейшей визуализации на панели приборов.
Главные актеры: Блок управления
Второстепенные актеры: Нет
Предусловия: Установлено WebSocket-соединение между блоком управления и системой
Основной поток: 1. Блок управления (ControlBlock) формирует телеметрические данные. 2. Данные сериализуются в формат JSON. 3. Система принимает данные через WebSocket. 4. Данные передаются в логику панели (useDashboardLogic). 5. Визуальные компоненты получают обновленные значения.
Постусловия: Система успешно получает и обрабатывает параметры автомобиля в режиме реального времени
Альтернативные потоки: Нет

Приложение Б. Листинги кода

Листинг 1 – Инициализация WebSocket-соединения в клиенте

```
useEffect(() => {
  socket = new WebSocket("ws://localhost:8080");
  socket.onmessage = (event) => {
    const data = JSON.parse(event.data);
    setSpeed(data.speed);
    setRpm(data.rpm);
    setGear(data.gear);
    setFuelLevel(data.fuel);
    setTemperature(data.temp);
    setVoltage(data.voltage);
    setMileage(data.mileage);
  };
  return () => socket?.close();
}, []);
```

Листинг 2 – Анимация сглаживания стрелки

```
useEffect(() => {
  const animate = () => {
    setDisplaySpeed(prev => {
      const delta = speed - prev;
      const step = delta * 0.2;
      return Math.abs(delta) < 0.1 ? speed : prev + step;
    });
    animationRef.current = requestAnimationFrame(animate);
  };
  animationRef.current = requestAnimationFrame(animate);
  return () => cancelAnimationFrame(animationRef.current!);
}, [speed]);
```

Листинг 3 – Пример JSON-пакета от ControlBlock

```
{
  "speed": 35,
  "rpm": 2100,
  "gear": "3",
  "fuel": 28.5,
  "temp": 91.2,
  "voltage": 13.8,
  "mileage": 15234.6
}
```

Листинг 4 – Обработка полученных данных в хуке useDashboardLogic

```
socket.onmessage = (event) => {
  const data = JSON.parse(event.data);
  setSpeed(data.speed);
  setRpm(data.rpm);
  setGear(data.gear);
  setFuelLevel(data.fuel);
  setTemperature(data.temp);
  setVoltage(data.voltage);
  setMileage(data.mileage);
};
```